

Introducción

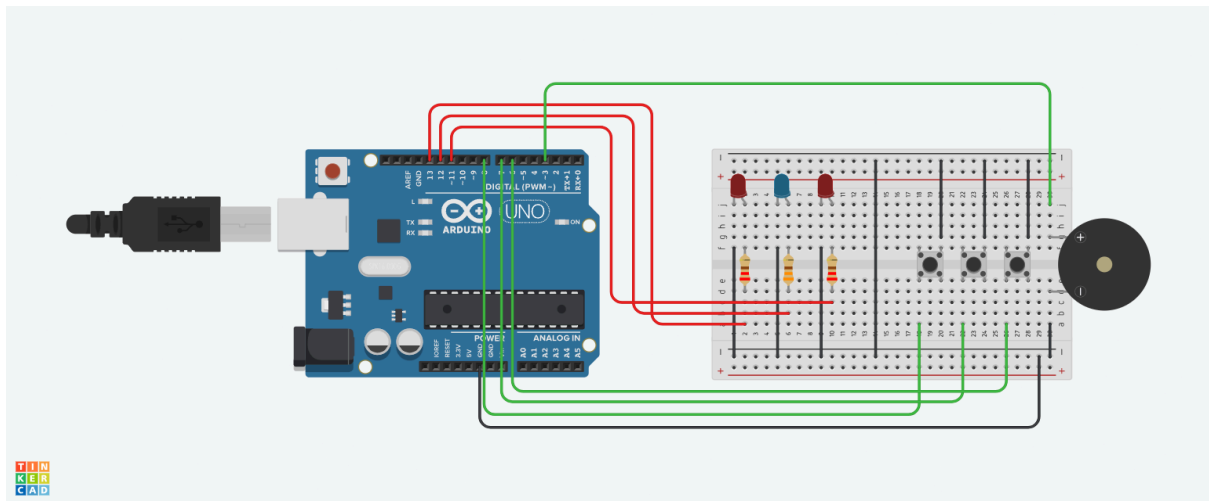
Después del avance de la sesión pasada, el objetivo de este circuito era tomar lo construido y agregarle un buzzer pasivo que emite una frecuencia cada vez que el usuario presiona el pulsador correcto. Otra adición que se le hizo al circuito, fue la creación de dos contadores diferentes; uno para medir las pulsaciones correctas del usuario y otro para reiniciar el juego en caso que el usuario se tarde más de cinco segundos en presionar el pulsador correcto.

Metodología

Materiales

- Arduino Uno R3 SMD
- Protoboard
- 2 LEDs rojos
- 1 LED azul
- 2 resistores de 220 Ω
- 1 resistor de 330 Ω
- 3 pulsadores
- Cables jumper
- Buzzer pasivo

Para este circuito se utilizó un Arduino para poder ejecutar el programa y un protoboard para realizar todas las conexiones necesarias. Para las conexiones de los LEDs se utilizaron tres resistencias; para los LEDs rojos la resistencia fue de 220 Ω y para el LED azul fue de 330 Ω . Se usó una resistencia más grande en el LED azul porque este tipo de luz consume más voltaje y es más sensible, así se evita que el LED brille demasiado o incluso que se queme. No se utilizaron resistencias para los pulsadores, puesto que estos se conectaron usando la función de Input PULLUP (la cual utiliza una resistencia interna).



Los componentes físicos fueron conectados conforme a la esquemática anterior. Se escogieron siete pines digitales para poder leer y mandar señales a los componentes: los pines 13, 12 y 11 fueron utilizados para mandar señales a los LEDs; los pines 8, 7 y 6 para leer el estado de los pulsadores, y el pin 3 para mandar señales al buzzer. Como ya fue mencionado, los pulsadores fueron conectados en modo PULL UP mientras que los LEDs y el buzzer en modo OUTPUT. Cabe mencionar como todos los componentes comparten la misma tierra, a pesar de estar en ramas independientes. Las relaciones entre componentes fueron determinadas a través del código.

Código utilizado

```

1  int leds[] = {13, 12, 11};
2  int botones[] = {8, 7, 6};
3  int buzzer = 3;
4  int ledActual;
5  int ContadorBoton = 0;
6  unsigned long tiempoInicio = 0;
7  bool timerActivo = false;
8  void setup() {
9      Serial.begin(9600);
10     pinMode(buzzer, OUTPUT);
11     for (int i = 0; i < 3; i++) {
12         pinMode(leds[i], OUTPUT);
13     }
14     for (int i = 0; i < 3; i++) {
15         pinMode(botones[i], INPUT_PULLUP);
16     }
17     randomSeed(analogRead(A0));
18     ledActual = random(0, 3);
19     digitalWrite(leds[ledActual], HIGH);
20 }
21 void loop() {
22     if (timerActivo && millis() - tiempoInicio > 5000) {
23         Serial.println("Tiempo agotado. Programa terminado.");
24         for (int i = 0; i < 3; i++) {
25             digitalWrite(leds[i], LOW);
26         }
27         while (true) {
28             ;
29         }
30         for (int i = 0; i < 3; i++) {
31             if (digitalRead(botones[i]) == LOW) {
32                 if (i == ledActual) {
33                     tiempoInicio = millis();
34                     timerActivo = true;
35                     digitalWrite(leds[ledActual], LOW);
36                     tone(buzzer, 440);
37                     delay(100);
38                     noTone(buzzer);
39                     ContadorBoton = ContadorBoton + 1;
40                     Serial.print("Contador de boton: ");
41                     Serial.println(ContadorBoton);
42                     int nuevoLED;
43                     do {
44                         nuevoLED = random(0, 3);
45                     } while (nuevoLED == ledActual);
46                     ledActual = nuevoLED;
47                     digitalWrite(leds[ledActual], HIGH);
48                     delay(200);
49                 }
50             }
51         }
52     }
}

```

- Se utilizó inteligencia artificial para auxiliar en el proceso de desarrollo del código.

Explicacion del codigo

```

1  int leds[] = {13, 12, 11};
2  int botones[] = {8, 7, 6};
3  int buzzer = 3;
4  int ledActual;
5  int ContadorBoton = 0;
6  unsigned long tiempoInicio = 0;
7  bool timerActivo = false;

```

Primeramente, se definen los arreglos y variables que se utilizarán más adelante. Se utilizan arreglos para poder vincular un LED con un pulsador en base a un solo valor, sin importar

las diferencias de entradas. De esta forma, si se llama al valor 0 de cada arreglo, se llamará a ambas entradas correctamente (la del LED y la del pulsador). Por otro lado, la variable "ledActual" servirá para trabajar con el LED prendido al momento de lectura. Adicionalmente, se creó la variable buzzer para poder emitir frecuencias desde el buzzer pasivo del circuito como salida. También se creó la variable de ContadorBoton para poder registrar el número de pulsaciones correctas de una partida y, por último, las variables tiempoInicio y timerActivo son utilizadas para poder contar el tiempo previo a la primera pulsación y activar el timer de 5 segundos respectivamente. La definición de tiempoInicio es unsigned long, ya que esta definición permite contar desde el 0 hasta 4,294,967,295, algo similar al contador interno del arduino "millis()". En contraste, la definición bool para timerActivo solo revisa si la variable es "falsa" o "verdadera". El valor de timerActivo empieza en "falso".

```
8 void setup() {
9   Serial.begin(9600);
10  pinMode(buzzer,OUTPUT);
11  for (int i = 0; i < 3; i++) {
12    pinMode(leds[i], OUTPUT);
13  }
14  for (int i = 0; i < 3; i++) {
15    pinMode(botones[i], INPUT_PULLUP);
16  }
```

Dentro del setup del sketch, se utiliza el operador "void" el cuál indica que las funciones "setup" y "loop" no regresan ningún valor. Primeramente, se define la velocidad de transmisión de datos a 9600 baudios con la función Serial.begin(9600). Después se establece el pin del buzzer (pin 3) como un valor de salida. Posteriormente, se define al arreglo de los leds como valores de salida por medio de "OUTPUT" y el arreglo de los pulsadores como entradas a través de la constante "INPUT_PULLUP".

```
17   randomSeed(analogRead(A0));
18   ledActual = random(0, 3);
19   digitalWrite(leds[ledActual], HIGH);
20 }
```

Posteriormente, se utiliza la función randomSeed para después empezar a generar valores aleatorios. La semilla permite que estos valores siempre sean aleatorios al iniciarse el programa. Después de esto se genera un valor aleatorio (dentro del rango del arreglo de los LED) para la variable ledActual, la cual es inmediatamente utilizada para prender un LED. Esta sección del código se encarga únicamente de prender el primer LED y asegurar que este y todos los que le sigan sean aleatorios.

```
21 void loop() {
22   if (timerActivo && millis() - tiempoInicio > 5000) {
23     Serial.println("Tiempo agotado. Programa terminado.");
24     for (int i = 0; i < 3; i++) {
25       digitalWrite(leds[i], LOW);
26     }
27     while (true) {
28     }
29   }
```

Iniciando el void loop, este empieza en una estructura lógica if que va a ser ejecutada si la variable timerActivo es verdadera y si la diferencia entre el valor de millis() y tiempoInicio

es mayor que 5000 milisegundos. En esencia, este bloque se encarga de “terminar” el programa si es que el usuario se tarda más de cinco segundos en presionar el pulsador correcto. Sin embargo, dado que `timerActivo` empieza como “falso” y solo se vuelve verdadero tras la primera presión correcta de un pulsador, esta condición no se ejecuta al iniciarse el programa. Es decir, el usuario se puede tardar todo lo que quiera para presionar el primer pulsador, y solo tiene que ser rápido en pulsaciones posteriores. Ahora, en más detalle, si es que el usuario se tarda más de cinco segundos en presionar el pulsador correcto (y no es la primera vez), el programa va a mostrar un mensaje de fin al usuario, utilizando la función `serial.println`. Posteriormente, se abre una estructura lógica “for” para poner en LOW a todos los pines que suplen corriente a los LEDs, y se utiliza “while” con la condición verdadera para poder evitar que el programa se siga ejecutando y de esta manera se termine el juego.

```
30     for (int i = 0; i < 3; i++) {
31         if (digitalRead(botones[i]) == LOW) {
32             if (i == ledActual) {
33                 tiempoInicio = millis();
34                 timerActivo = true;
35                 digitalWrite(leds[ledActual], LOW);
36                 tone(buzzer, 440);
37                 delay(100);
38                 noTone(buzzer);
39                 ContadorBoton = ContadorBoton + 1;
40                 Serial.print("Contador de boton: ");
41                 Serial.println(ContadorBoton);
42                 int nuevoLED;
43                 do {
44                     nuevoLED = random(0, 3);
45                 } while (nuevoLED == ledActual);
46                 ledActual = nuevoLED;
47                 digitalWrite(leds[ledActual], HIGH);
48                 delay(200);
49             }
50         }
51     }
52 }
```

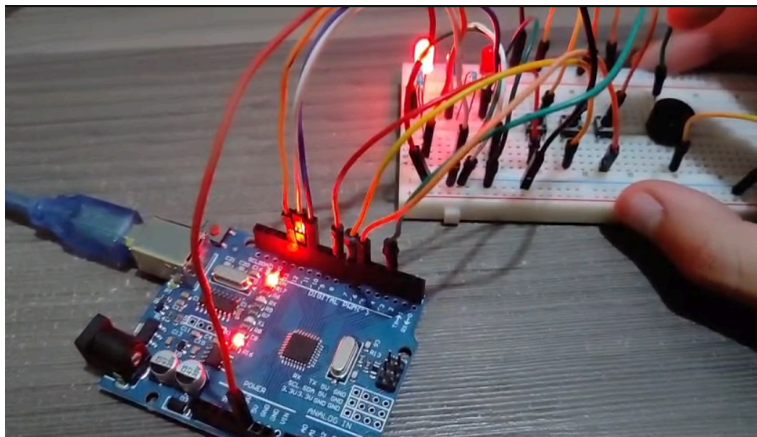
Continuando, se abre un bucle for para revisar el estado de los 3 pulsadores. Primeramente, se evalúa si la señal recibida de un pulsador corresponde en cuanto a posición en un arreglo a la del LED activamente prendido. En caso de que no, no sucede nada. En caso de que si, primero se actualiza el valor de `tiempoInicio` al valor actual de `millis` (el cual va contando desde que el programa se inicia). Esto permite que se compare el tiempo transcurrido desde la pulsación para su uso en la sección pasada. Después, se cambia el valor de `timerActivo` a “verdadero”, lo cual como ya fue mencionado solo sirve para que solo se empiecen a contar los 5 segundos después de la primera pulsación. Dado que el valor de `timerActivo` no regresa a “falso” nunca, esta línea solo es importante la primera vez que se ejecuta. Después de esto, se apaga el LED activamente prendido, se indica al buzzer que haga un ruido de 440 Hz, se hace una pequeña pausa, se detiene el ruido del buzzer y se agrega 1 al valor de `ContadorBoton`. Este valor luego es impreso a través de `Serial.print` en el monitor serial para ir llevando cuenta del número de pulsaciones correctas (más que nada para depurar el programa). Después de esto, el programa entra en el bucle “do while”, el cual selecciona el siguiente LED a prenderse evitando que este se repita del previo y lo prende. Para finalizar, se agrega un pequeño “delay (200)” esto para que no haya un rebote con los botones al momento que son presionados. Como los botones, al momento de

presionarlo hay veces en la que la persona es muy lenta, esto hace que se manden varias señales, con este delay se le da el tiempo suficiente al botón se estabilice y evite que el arduino cuente dos puntos con una sola pulsación.

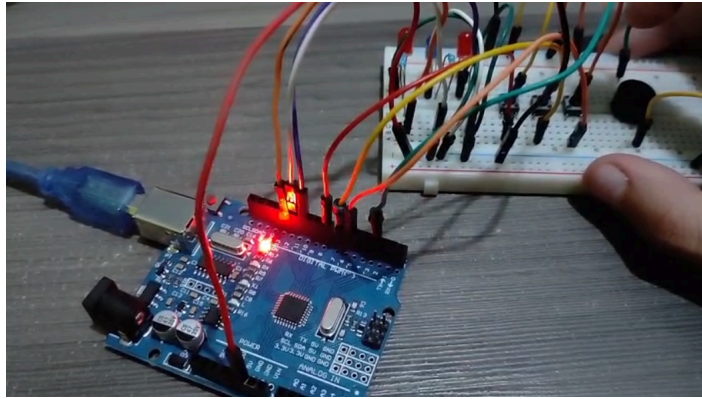
Resultados

Una vez el sistema conectado y el programa subido al Arduino, se probó su funcionamiento. Los resultados de este dieron lo siguiente:

- Cada vez que presionas el botón correcto del LED encendido, la luz se apaga, el buzzer emite una frecuencia de 440 hz por 100 ms y se enciende un nuevo LED al azar, asegurandose que no se repita el mismo que se acaba de apagar. Al mismo tiempo, el contador de 5 segundos se vuelve a reiniciar.



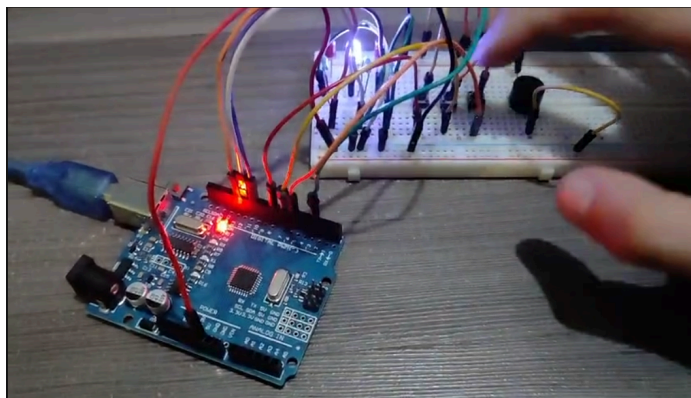
- Si pasan más de 5 segundos sin presionar el botón correcto, todos los LEDs se apagan y muestra en la terminal el mensaje "Tiempo agotado. Programa terminado". quedando registrado arriba el último puntaje que se alcanzó.



Monitor en serie

```
Contador de boton: 1
Contador de boton: 2
Contador de boton: 3
Contador de boton: 4
Contador de boton: 5
Contador de boton: 6
Contador de boton: 7
Contador de boton: 8
Contador de boton: 9
Contador de boton: 10
Contador de boton: 11
Contador de boton: 12
Tiempo agotado. Programa terminado.
```

- Las funcionalidades previas se mantuvieron para este nuevo proyecto.



Conclusión

Para concluir, el proyecto alcanzó todos los objetivos esperados. Adicional a todas las funciones pasadas del proyecto, ahora se registran todas las pulsaciones correctas del usuario por partida, se emite una frecuencia cada vez que el usuario hace una pulsación correcta y el juego termina al cabo de cinco segundos de no registrar la pulsación correspondiente al LED prendido.

Por el lado del ensamblaje físico, el proceso fue rápido y sin mayores complicaciones, ya que la única adición relevante fue conectar el buzzer a tierra y a su correspondiente pin digital. Sin embargo, la parte de programación requirió bastantes cambios y la integración de conceptos de código que no habíamos manejado anteriormente, lo que representó el principal reto del proyecto.

Los aprendizajes de este circuito fueron el uso de las nuevas definiciones para las variables (unsigned long y bool), la función del Arduino "milis()" y los operadores lógicos dentro de las condiciones para las estructuras lógicas. En resumen, todo lo relacionado al registro y medición del tiempo y lo relacionado a cuestiones de sonido tuvo que ser aprendido e implementado a esta actualización del proyecto.

Referencias

OpenAI. (2026, 27 de agosto). *Programar juego de reacción* [Chat de IA generativa].

ChatGPT.

<https://chatgpt.com/share/6a94ecd6-1cac-83e8-bdbd-6b8118fc2b4d>